Applied NLP

DATA 641

Lecture 8 — Loopy Recurrent Neural Networks

*Practical Natural Language Processing: A Comprehensive Guide to Building Real-World NLP Systems 1st Edition, Sowmya Vajjala, Bodhisattwa Majumder, Anuj Gupta, Harshit Surana, O' Reilly and Natural Language Processing in Action, Understanding, analyzing, and generating text with Python, Hobson Lane, Cole Howard, Hannes Hapke, First edition, MANNING

*Notes for this lecture are generated by the online lecture notes: https://lena-voita.github.io/nlp_course

*Useful video on recurrent neural networks: https://www.youtube.com/watch?v=AsNTP8Kwu80&t=667s&ab_channel=StatQuestwithJoshStarmer

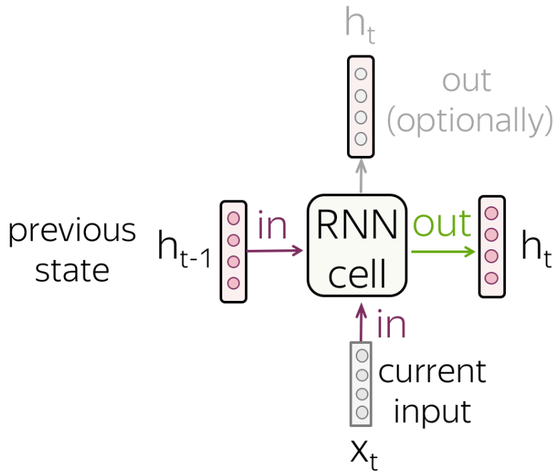
Understanding recurrent neural networks

- A major characteristic of all neural networks we have seen so far, such as densely connected networks and CNNs, is that they have no memory. Each input shown to them is processed independently, with no state kept between inputs.

- In contrast, as we are reading the present sentence, we are processing it word by word while keeping memories of what came before; this gives us a fluid representation of the meaning conveyed by this sentence. Biological intelligence processes information incrementally while maintaining an internal model of what it is processing, built from past information and constantly updated as new information comes in.

RNN cell

- At each step, a **recurrent neural network (RNN)** receives a new input vector (e.g., token embedding) and the previous network state (which, hopefully, encodes all previous information). Using this input, the RNN cell computes the new state which it gives as output. This new state now contains information about both current input and the information from previous steps.



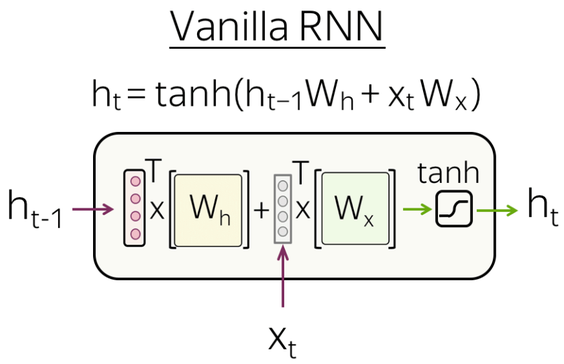
- The state of the RNN is reset between processing two different, independent sequences (such as two samples in a batch), so you still consider one sequence to be a single data point: a single input to the network. What changes is that this data point is no longer processed in a single step; rather, the network internally loops over sequence elements.

Vanilla RNN

The simplest recurrent network, Vanilla RNN, transforms h_{t-1} and x_t linearly, then applies a non-linearity (most often, the $\tanh$ function)

$$h_t = \tanh(h_{t-1}W_h + x_t W_t) \tag{1}$$

Vanilla RNNs suffer from the vanishing and exploding gradients problem. To alleviate this problem, more complex recurrent cells (e.g., LSTM, GRU, etc) perform several operations on the input and use gates.



RNN runtime performance

- Recurrent models with very few parameters, tend to be significantly faster on a multicore CPU than on `GPU`, because they only involve small matrix multiplications, and the chain of multiplications is not well parallelizable due to the presence of a `for` loop. But larger `RNNs` can greatly benefit from a GPU runtime.

- When using Keras LSTM or GRU layers with default settings on a GPU, they utilize cuDNN kernels, an optimized NVIDIA-provided implementation of the algorithm. However, cuDNN kernels have limitations—they are fast but inflexible. If you attempt anything unsupported by the default kernel, it can result in a significant performance drop. For instance, adding recurrent dropout to LSTM and GRU layers requires falling back to the regular TensorFlow implementation, which is notably slower on GPU (despite having the same computational cost).

- As a way to speed up your RNN layer when you can't use cuDNN, you can try unrolling it. Unrolling a `for` loop consists of removing the loop and simply inlining its content N times. In the case of the `for` loop of an RNN, unrolling can help TensorFlow optimize the underlying computation graph. However, it will also considerably increase